

CSC 402 – Mobile Application Programming

Lab 2 – Part B: Jetpack Compose

Tasks 6–11

Student Name: Elaf Hassan Al-Nutayf

Student ID: 2240005931

Task 6 – Welcome Screen

The WelcomeScreen composable was created using Jetpack Compose.

The screen uses Column, Box, Card, Text, Spacer, and Modifier to build the complete interface.

The application was run successfully on the Android emulator.

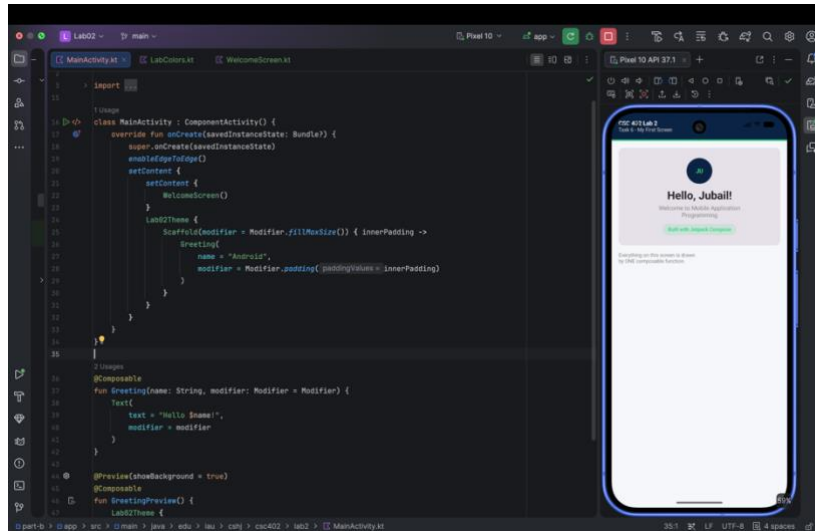


Figure 1. Task 6 running on the Android emulator.

Task 7 – Student ID Screen

The Student ID screen was implemented using Row and Column layouts.

A reusable StatItem composable was used to display GPA, credits, and status.

The screen was tested successfully on the Android emulator.

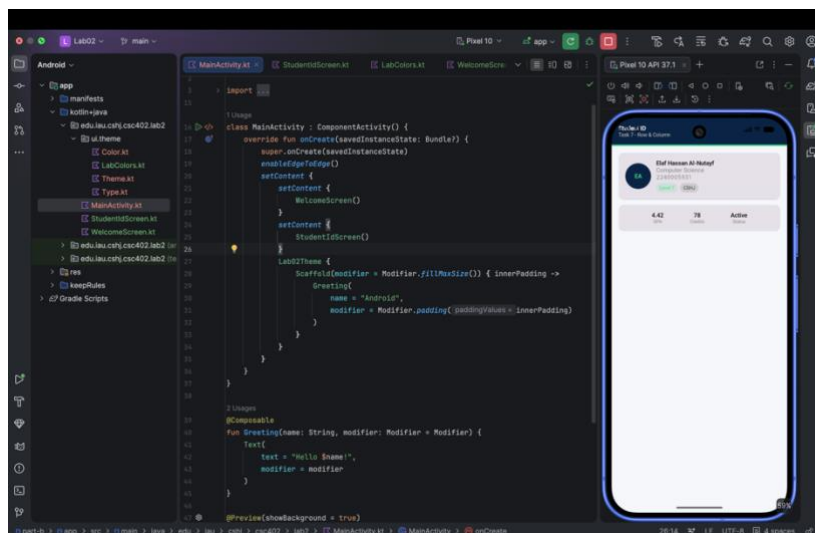


Figure 2. Task 7 Student ID screen running on the emulator.

Task 8 – Attendance Counter

The AttendanceCounter composable uses remember and mutableStateOf to store the attendance count.

The interface updates automatically when the count changes.

Two emulator screenshots are shown below to demonstrate different states of the same screen.

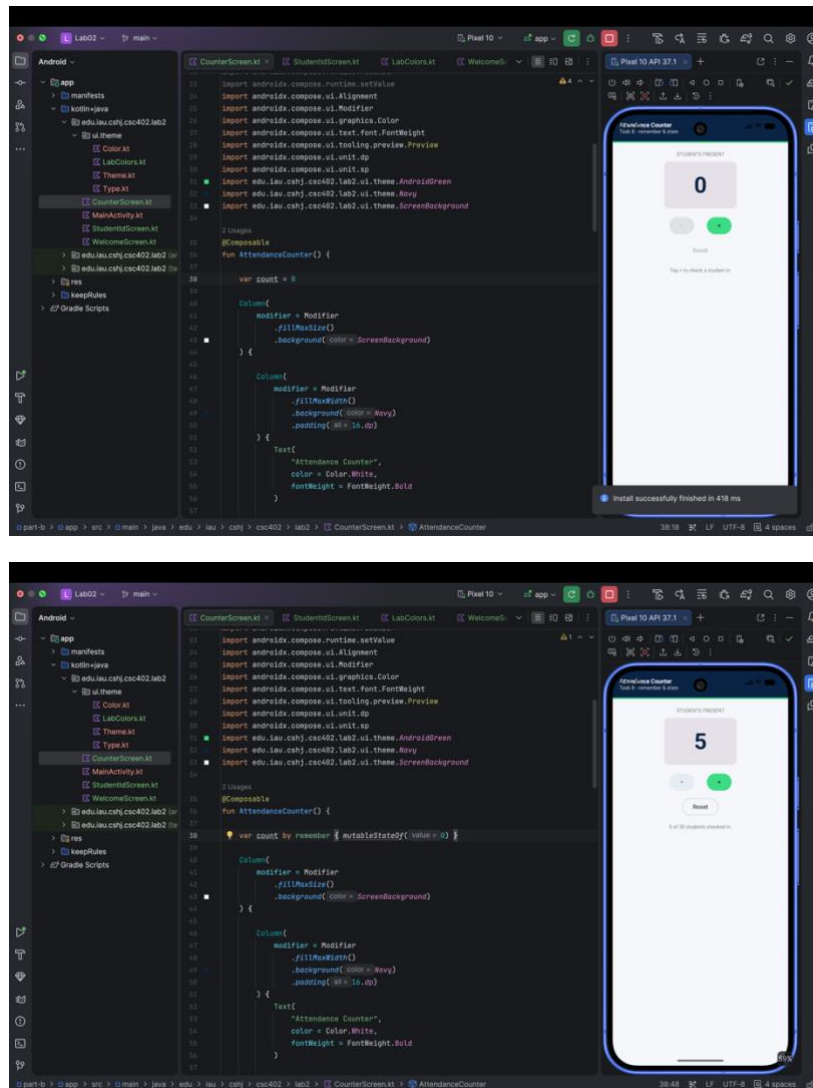


Figure 3. Task 8 initial state and updated attendance state.

Q6. Why did the screen not change when a plain variable was used?

With a plain variable, changing the value does not create observable Compose state, so Compose is not notified that the UI needs to update. `mutableStateOf` creates observable state, and when its value changes, recomposition occurs and the new value is displayed. `remember` keeps that state across recompositions.

Task 9 – Greeting Card

The GreetingCard screen uses an OutlinedTextField and Compose state to store the entered name.

The button is enabled only when the text field is not blank.

After submission, the greeting card updates using the submitted name and calculates the number of letters dynamically.

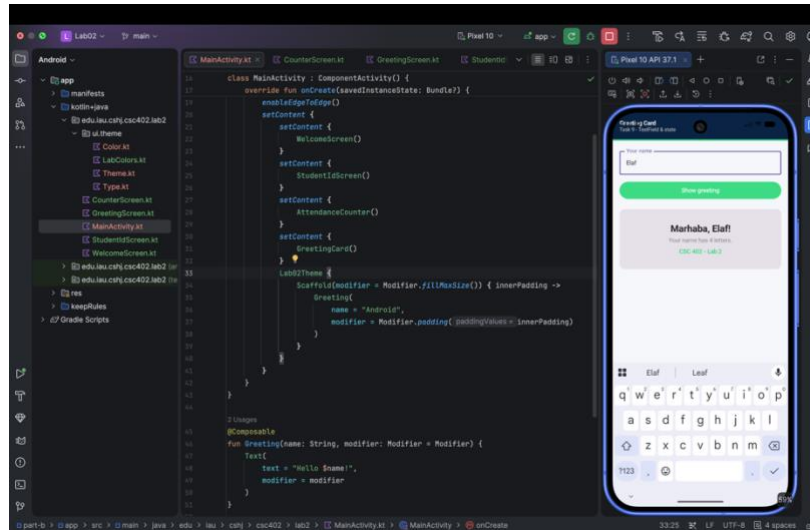


Figure 4. Task 9 greeting card after entering a name.

Task 10 – Course List with LazyColumn

The course list was implemented using LazyColumn and a reusable CourseRow composable.

The courses are stored as Course data objects and displayed as scrollable cards.

Each row shows the course code, title, days, and credit hours.

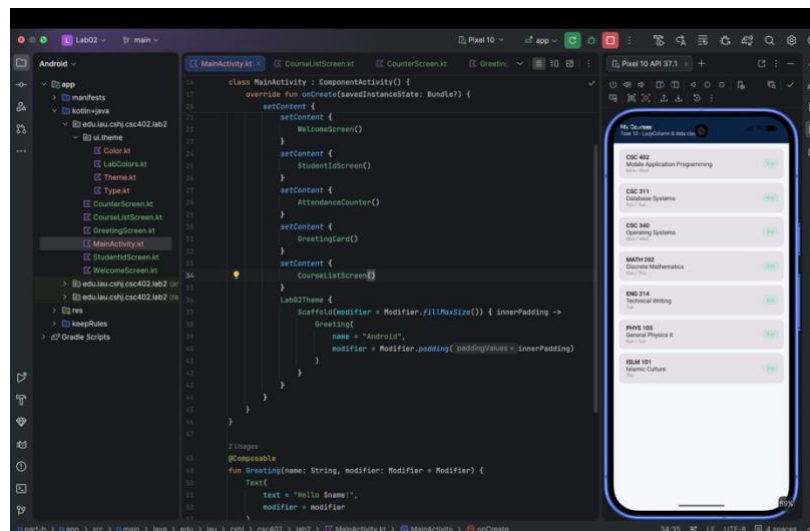


Figure 5. Task 10 course list running on the emulator.

Q7. Why is LazyColumn better than Column with verticalScroll for a large list?

LazyColumn only composes the items that are currently visible or close to the visible area. This makes it more efficient for long lists because it avoids creating every item at once. A Column with verticalScroll would compose all items, which would use more memory and processing for a large number of courses.

Task 11 – Team Task Tracker

The Team Task Tracker combines data classes, mutable state, derived values, LazyColumn, and a stateless child composable.

The Done, Open, and Total values are calculated from the task list rather than being hard-coded.

Each task is displayed using the reusable TaskRow composable.

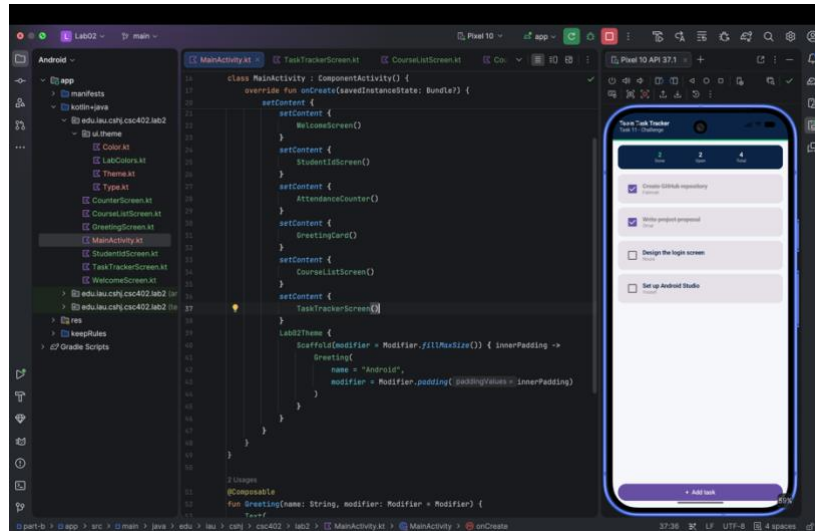


Figure 6. Task 11 Team Task Tracker running on the emulator.

Q8. Why should TaskRow not keep its own isDone state?

TaskRow should not keep a separate isDone state because the parent task list is the source of truth used to calculate the Done, Open, and Total values. If the row changed only its own local state, the parent list would not know about the change and the summary could become incorrect. State hoisting keeps the row and the summary synchronized.

AI Use & Disclosure Log

Tool	How it was used	What I changed / verified
ChatGPT	Used as a pair programmer to explain requirements, suggest code structure, and troubleshoot Android Studio issues.	I reviewed the code, tested it on the emulator, adjusted layout and spacing, and verified the final output.